lecture_11

August 24, 2026

1 Lecture 11: Transportation Engineering Problems - II

```
{note} Lecture 10 introduced the network-LP toolkit - nodes, arcs, flow
conservation - through the Maximum Flow and Least-Cost Path problems, each with
a single source and a single sink. This lecture extends the same toolkit to
networks with multiple sources and multiple destinations: the **Transportation
Problem** (sources ship directly to destinations) and the **Transshipment
Problem** (sources ship via intermediate hubs before reaching destinations). The
underlying LP machinery - formulation, Excel Solver, Python SciPy, sensitivity
analysis, duality - is unchanged; only the network structure grows richer.
```

Learning Objectives

By the end of this notebook, you will be able to: 1. Represent multi-source, multi-destination transportation networks as graphs, and translate them into LP formulations for the Transportation Problem and the Transshipment Problem. 2. Solve network LPs using Excel Solver and Python SciPy. 3. Interpret shadow prices, reduced costs, and complementary slackness in a transportation/transshipment context (source-capacity bottlenecks, redundant routes, hub utilisation) to make informed distribution and logistics-network decisions.

Prerequisites: LP Formulation (Lecture 3), Excel Solver (Lecture 5), Python SciPy Method (Lecture 6), Sensitivity Analysis (Lecture 8), Duality (Lecture 9), Network Problems - I (Lecture 10)

Estimated time: 90 minutes (including in-class exercises)

1.1 Why Transportation and Transshipment Networks?

Lecture 10's networks had a single origin and a single destination. Real distribution networks rarely look like that:

- MTC Chennai asks: *we have buses sitting idle at two depots and three terminals need buses today — how many buses should each depot send to each terminal to minimize total relocation cost?* — a **Transportation Problem**.
- A freight operator asks: *goods leave two source cities, pass through a consolidation hub, and are delivered to two destination cities — how much should flow along each leg to minimize total shipping cost?* — a **Transshipment Problem**.

Both are still network-flow LPs governed by flow conservation, exactly as in Lecture 10. What changes is the shape of the network: instead of one source and one sink connected by a chain of intermediate nodes, we now have several supply nodes and several demand nodes, optionally connected through intermediate trans-shipment nodes.

1.2 Transportation Problem vs. Transshipment Problem

	Transportation Problem	Transshipment Problem
Sources	m supply nodes, each with a fixed supply s_i	Same
Destinations	n demand nodes, each with a fixed demand d_j	Same
Intermediate nodes	None — every arc goes directly source → destination	One or more **trans-shipment nodes**, with pure flow-conservation (no supply or demand of their own)
Arcs	(i, j) for every source-destination pair	Source → hub, hub → hub, hub → destination — not all pairs need be connected

`{tip} A Transportation Problem is simply a Transshipment Problem with zero intermediate nodes. Lecture 10's Least-Cost Path problem, in turn, is a Transshipment Problem with exactly one unit of supply and one unit of demand. All of Lectures 10-11 are variations on the same flow-conservation idea - only the network topology and the supply/demand pattern change.`

1.3 The Transportation Problem

1.3.1 Problem Statement

Given m sources, each with a fixed supply s_i, and n destinations, each with a fixed demand d_j, and a cost c_{ij} per unit shipped directly from source i to destination j, find the shipment quantities x_{ij} that meet all demand from available supply at minimum total cost.

1.3.2 LP Formulation

Decision variables: $x_{ij} \geq 0$ = units shipped from source i to destination j, for every source-destination pair.

Objective: minimize total shipping cost,

$$\min \quad z = \sum_{i \in \mathcal{S}} \sum_{j \in \mathcal{D}} c_{ij}\, x_{ij}$$

Subject to:

$$\textbf{(Supply)} \qquad \sum_{j \in \mathcal{D}} x_{ij} \leq s_i \qquad \forall\, i \in \mathcal{S}$$

$$\textbf{(Demand)} \qquad \sum_{i \in \mathcal{S}} x_{ij} \geq d_j \qquad \forall\, j \in \mathcal{D}$$

$$x_{ij} \geq 0 \qquad \forall\, i \in \mathcal{S}, j \in \mathcal{D}$$

where $\mathcal{S}$ is the set of sources and $\mathcal{D}$ is the set of destinations.

{note} **Balanced vs. Unbalanced**: if $\sum_i s_i = \sum_j d_j$, the problem is balanced and both supply and demand constraints hold with equality at the optimum (every unit of supply is used, every unit of demand is met). If total supply exceeds total demand, supply constraints stay as $\leq$ and some supply is left unused; this course works exclusively with balanced transportation problems unless stated otherwise.

This is, once again, a flow-conservation network: supply nodes are pure producers, demand nodes are pure consumers, and there are no intermediate nodes — every arc runs directly from a source to a destination.

1.4 In-Class Exercises

1.4.1 Exercise 1

MTC Chennai has idle buses sitting at two depots and needs to relocate them to three terminals to meet today's service requirements.

	Depot 1 (Ambattur)	Depot 2 (Tambaram)	Supply (buses)
Terminal A	4,000/bus	5,000/bus	—
Terminal B	6,000/bus	4,000/bus	—
Terminal C	8,000/bus	3,000/bus	—
Demand (buses)	—	—	—

Supply: Depot 1 has 30 buses available, Depot 2 has 40 buses available. Demand: Terminal A needs 20 buses, Terminal B needs 25 buses, Terminal C needs 25 buses.

Minimize total relocation cost.

Step 1 — Formulate the LP Let x_{ij} denote the number of buses sent from Depot $i \in \{1, 2\}$ to Terminal $j \in \{A, B, C\}$.

$$\min \quad z = 4x_{1A} + 6x_{1B} + 8x_{1C} + 5x_{2A} + 4x_{2B} + 3x_{2C}$$

$$
\begin{aligned}
\textbf{(Supply)} \quad & x_{1A} + x_{1B} + x_{1C} \leq 30 \\
& x_{2A} + x_{2B} + x_{2C} \leq 40 \\
\textbf{(Demand)} \quad & x_{1A} + x_{2A} \geq 20 \\
& x_{1B} + x_{2B} \geq 25 \\
& x_{1C} + x_{2C} \geq 25 \\
& x_{ij} \geq 0 \quad \forall i, j
\end{aligned}
$$

Note: Total supply ($30 + 40 = 70$) equals total demand ($20 + 25 + 25 = 70$) — this is a balanced transportation problem, so every supply and demand constraint is written as an equality.

Step 2 — Excel Spreadsheet Layout

```
     A                      B          C          D          E
 1   MTC BUS RELOCATION - TRANSPORTATION PROBLEM
 2
 3   Cost ( '000/bus)      Term A     Term B     Term C     Supply
 4   Depot 1 (Ambattur)    4          6          8          30
 5   Depot 2 (Tambaram)    5          4          3          40
 6   Demand                20         25         25
 7
 8   Decision Variables    Term A     Term B     Term C
 9   Depot 1               0          0          0          ← Solver changes these
10   Depot 2               0          0          0          ← Solver changes these
11
12   Objective
13   Total Cost ( '000)    =SUMPRODUCT(B4:D5, B9:D10)
14
15   Constraints           LHS                RHS
16   Supply Depot 1        =SUM(B9:D9)        30
17   Supply Depot 2        =SUM(B10:D10)      40
18   Demand Term A         =SUM(B9:B10)       20
19   Demand Term B         =SUM(C9:C10)       25
20   Demand Term C         =SUM(D9:D10)       25
```

Step 3 — Solver Configuration

- **Set Objective**: B13 → **Min**
- **Variable Cells**: B9:D10
- **Constraints**:
  - B16 <= C16
  - B17 <= C17
  - B18 >= C18
  - B19 >= C19
  - B20 >= C20

  - B9:D10 $\geq$ 0
- **Method**: Simplex LP → Solve

Step 4 — Expected Results & Interpretation

	Terminal A	Terminal B	Terminal C
Depot 1 (Ambattur)	20	10	0
Depot 2 (Tambaram)	0	15	25

Total relocation cost = 2,75,000

> **Managerial insight**: Depot 1 (Ambattur) covers all of Terminal A's demand and part of Terminal B's, while Depot 2 (Tambaram) covers the rest of Terminal B and all of Terminal C — Tambaram is much cheaper for Terminal C (3,000 vs 8,000/bus), so the solver routes every Terminal C bus from there. No depot ships to every terminal; the optimal pattern naturally avoids the most expensive routes (Depot 1 → Terminal C is never used).

1.5 The Transshipment Problem

1.5.1 Problem Statement

Given m sources with fixed supply s_i, n destinations with fixed demand d_j, and one or more **transshipment nodes** that neither supply nor demand goods but simply pass flow through, find the shipment quantities on every arc that meet all demand from available supply at minimum total cost.

1.5.2 LP Formulation

Decision variables: $x_{ij} \geq 0$ = flow on arc (i, j), for every $(i, j) \in \mathcal{A}$ — arcs may run source → hub, hub → hub, or hub → destination (and occasionally source → destination directly, if such an arc exists).

Objective: minimize total shipping cost,

$$\min \quad z = \sum_{(i,j)\in\mathcal{A}} c_{ij}\, x_{ij}$$

Subject to:

$$\textbf{(Supply)} \qquad \sum_{j:(i,j)\in\mathcal{A}} x_{ij} \le s_i \qquad \forall\, i \in \mathcal{S}$$

$$\textbf{(Transshipment)} \qquad \sum_{i:(i,j)\in\mathcal{A}} x_{ij} - \sum_{k:(j,k)\in\mathcal{A}} x_{jk} = 0 \qquad \forall\, j \in \mathcal{N} \setminus \{o, d\}$$

$$\textbf{(Demand)} \qquad \sum_{i:(i,j)\in\mathcal{A}} x_{ij} \ge d_j \qquad \forall\, j \in \mathcal{D}$$

$$x_{ij} \ge 0 \qquad \forall\, (i,j) \in \mathcal{A}$$

where $\mathcal{S}$ is the set of sources, $\mathcal{T}$ the set of trans-shipment nodes, and $\mathcal{D}$ the set of destinations.

`{tip} The **Trans-shipment constraint** is exactly the flow-conservation constraint from Lecture 10 - "flow in = flow out" at every node that neither supplies nor demands goods. A Transportation Problem is the special case $\mathcal{T} = \emptyset$.`

1.5.3 Exercise 2

A freight operator ships goods from two source cities, Mumbai (S1) and Pune (S2), to two destination cities, Surat (D1) and Vadodara (D2). All shipments are consolidated through a hub at Nashik (T) — except for one direct bypass link, Pune → Surat, available at a premium rate.

Arc	From	To	Cost (100/tonne)
1	Mumbai (S1)	Nashik (T)	3
2	Pune (S2)	Nashik (T)	2
3	Pune (S2)	Surat (D1) — direct bypass	7
4	Nashik (T)	Surat (D1)	4
5	Nashik (T)	Vadodara (D2)	5

Supply: Mumbai has 40 tonnes available, Pune has 35 tonnes available. Demand: Surat needs 30 tonnes, Vadodara needs 25 tonnes.

Minimize total shipping cost.

Step 1 — Formulate the LP Let $x_{S1T}, x_{S2T}, x_{S2D1}, x_{TD1}, x_{TD2}$ denote flow (tonnes) on each arc.

$$\min \quad z = 3x_{S1T} + 2x_{S2T} + 7x_{S2D1} + 4x_{TD1} + 5x_{TD2}$$

$$
\begin{aligned}
\text{(Supply)} \quad & x_{S1T} \leq 40 \\
& x_{S2T} + x_{S2D1} \leq 35 \\
\text{(Trans-shipment at T)} \quad & x_{S1T} + x_{S2T} - x_{TD1} - x_{TD2} = 0 \\
\text{(Demand)} \quad & x_{S2D1} + x_{TD1} \geq 30 \\
& x_{TD2} \geq 25 \\
& x_{S1T}, x_{S2T}, x_{S2D1}, x_{TD1}, x_{TD2} \geq 0
\end{aligned}
$$

Step 2 — Solve with `scipy.optimize.linprog`

[2]:
```
# --- Exercise 2: solve the Mumbai/Pune-Nashik-Surat/Vadodara transshipment LP---

import numpy as np
from scipy.optimize import linprog

c = [3, 2, 7, 4, 5]                  # cost on x_S1T, x_S2T, x_S2D1, x_TD1,x_TD2

# Inequality constraints: supply caps
A_ub = [
    [1, 0, 0, 0, 0],                 # Supply at S1 (Mumbai): x_S1T <= 40
    [0, 1, 1, 0, 0],                 # Supply at S2 (Pune): x_S2T + x_S2D1 <= 35
    [0, 0, -1, -1, 0],                 # demand at D1 (Surat): -x_S2D1 - x_TD1>= -30
    [0, 0, 0, 0, -1],                  # demand at D2 (Vadodara): -x_TD2 >= -25
]
b_ub = [40, 35, -30, -25]

# Equality constraints: trans-shipment + demand
A_eq = [
    [1, 1, 0, -1, -1],               # trans-shipment at T: in = out
]
b_eq = [0]

bounds = [(0, None)] * 5

res = linprog(c, A_ub=A_ub, b_ub=b_ub, A_eq=A_eq, b_eq=b_eq, bounds=bounds,method='highs')

arc_names = ["x_S1T", "x_S2T", "x_S2D1", "x_TD1", "x_TD2"]
print("Optimal flows (tonnes):")
for name, val in zip(arc_names, res.x):
    print(f"  {name} = {val:.1f}")
print(f"\nMinimum cost z* =  {res.fun*100:,.0f}")
```

```
Optimal flows (tonnes):
  x_S1T = 20.0
  x_S2T = 35.0
  x_S2D1 = 0.0
  x_TD1 = 30.0
  x_TD2 = 25.0

Minimum cost z* =  37,500
```

Optimal solution: $x^*_{S1T} = 20$, $x^*_{S2T} = 35$, $x^*_{S2D1} = 0$, $x^*_{TD1} = 30$, $x^*_{TD2} = 25$, $z^* = 375$ (37,500).

All flow is routed through the Nashik hub; the direct Pune → Surat bypass is unused, even though it would save the hub leg. Mumbai supplies only 20 of its available 40 tonnes — it has spare capacity — while Pune's entire 35-tonne supply is consumed.

Step 3 — Sensitivity Analysis: Source Capacity and the Unused Bypass Arc

[3]:
```
# --- Exercise 2: shadow prices on supply capacity and reduced cost of the␣
 ↪bypass arc ---
print("Shadow prices on supply constraints:")
print(f"  Mumbai (S1) capacity : {res.ineqlin.marginals[0]:.2f}")
print(f"  Pune (S2) capacity   : {res.ineqlin.marginals[1]:.2f}")

print("\nReduced cost of each arc:")
for name, val in zip(arc_names, res.lower.marginals):
    print(f"  {name}: {val:.2f}")
```

```
Shadow prices on supply constraints:
  Mumbai (S1) capacity : -0.00
  Pune (S2) capacity   : -1.00

Reduced cost of each arc:
  x_S1T: 0.00
  x_S2T: 0.00
  x_S2D1: 1.00
  x_TD1: 0.00
  x_TD2: 0.00
```

Reading the output:

Quantity	Value	Interpretation
Shadow price — Mumbai capacity	0	Mumbai's 40-tonne capacity is not binding (only 20 tonnes used) — one more tonne of Mumbai capacity is worth nothing to this plan

Quantity	Value	Interpretation
Shadow price — Pune capacity	-1	Pune's 35-tonne capacity is binding — one more tonne of Pune capacity would *reduce* total cost by ₹100 (Pune is the cheaper source into the hub)
Reduced cost — x_{S2D1} (Pune → Surat bypass)	1	The bypass costs ₹100/tonne more than routing the same tonne via the Nashik hub — it would need to drop to ₹600/tonne (from ₹700) before the solver would use it

```
{note} This mirrors Lecture 10's node-potential logic exactly: an unused arc's
reduced cost tells the planner precisely how much cheaper that arc would need
to become before it joins the optimal solution. Here, the "almost competitive"
bypass arc (reduced cost of only 1, versus arcs with much larger reduced costs)
is the one worth watching if Pune-Surat tolls or fuel costs change.
```

> **Managerial insight**: Because Pune's capacity is the binding constraint, any investment in expanding Pune's dispatch capacity (extra loading bays, additional drivers) directly reduces total network cost — at the margin, ₹100 per additional tonne. Mumbai's spare 20 tonnes of capacity, by contrast, is not worth paying to expand under current demand.

1.6 Take-Away Exercises

Work through each exercise following these steps: 1. Draw the network. 2. Formulate the LP (decision variables, objective, constraints). 3. Solve using the specified tool (Excel Solver or Python SciPy). 4. Where indicated, carry out the sensitivity/duality analysis requested. 5. Record the optimal solution and write a one-paragraph managerial interpretation.

1.6.1 Exercise 1

A Kochi-based freight distributor ships goods from two warehouses to three retail clusters.

	Cluster 1	Cluster 2	Cluster 3
Warehouse 1	₹6,000/tonne	₹4,000/tonne	₹9,000/tonne
Warehouse 2	₹3,000/tonne	₹7,000/tonne	₹5,000/tonne

Supply: Warehouse 1 has 25 tonnes available, Warehouse 2 has 35 tonnes available.

Demand: Cluster 1 needs 15 tonnes, Cluster 2 needs 20 tonnes, Cluster 3 needs 25 tonnes.

Minimize total shipping cost.

Use Excel Solver.

1.6.2 Exercise 2

A Jaipur-based distributor ships from two warehouses to two retail clusters.

	Cluster 1	Cluster 2
Warehouse 1	5,000/tonne	8,000/tonne
Warehouse 2	6,000/tonne	4,000/tonne

Supply: Warehouse 1 has 40 tonnes available, Warehouse 2 has 30 tonnes available.

Demand: Cluster 1 needs 35 tonnes, Cluster 2 needs 35 tonnes.

Minimize total shipping cost.

Use Python SciPy.

1. Identify any source-destination pair with zero optimal flow, and report its reduced cost.
2. Verify complementary slackness for this pair — confirm that its primal flow is zero exactly because its dual constraint has positive slack (recall Lecture 9), and interpret what this means for the distributor (would this route ever be used, even at the margin?).

1.6.3 Exercise 3

A Delhivery-style courier network ships parcels from two source hubs to two destination cities, consolidated through a single intermediate sorting facility.

Arc	From	To	Cost (/parcel)
1	Source 1	Sorting Facility	4
2	Source 2	Sorting Facility	3
3	Sorting Facility	Destination 1	5
4	Sorting Facility	Destination 2	2

Supply: Source 1 has 30 parcels/hr available, Source 2 has 40 parcels/hr available.

Demand: Destination 1 needs 25 parcels/hr, Destination 2 needs 30 parcels/hr.

Minimize total shipping cost.

Use Excel Solver.

1.6.4 Exercise 4

A freight operator ships from two source cities to two destination cities through a single hub, with one direct bypass arc available from Source 1 to Destination 2.

Arc	From	To	Cost (100/tonne)
1	Source 1	Hub	2
2	Source 2	Hub	5
3	Source 1	Destination 2 — direct bypass	9
4	Hub	Destination 1	3
5	Hub	Destination 2	4

Supply: Source 1 has 35 tonnes available, Source 2 has 30 tonnes available.

Demand: Destination 1 needs 20 tonnes, Destination 2 needs 30 tonnes.

Minimize total shipping cost.

Use Python SciPy.

1. Report the shadow price of each source's capacity constraint and the reduced cost of the bypass arc.
2. For the bypass arc, state how much its cost would need to fall before it enters the optimal solution.

1.7 Circling Back

- **Lecture 3 (LP Formulation), Lecture 5 (Excel Solver), Lecture 6 (Python SciPy)**: Transportation and Transshipment Problems are solved with the exact same formulation and solver skills as every other LP in this course — only the constraint pattern (supply, transshipment, demand) is new.
- **Lecture 8 (Sensitivity Analysis)**: shadow prices on supply constraints identify which source's capacity is the binding bottleneck; reduced costs on unused arcs quantify exactly how much cheaper a route would need to become to enter the optimal plan.
- **Lecture 9 (Duality)**: complementary slackness explains *why* certain source-destination pairs carry zero flow at optimality — their dual constraint has slack, exactly as explored in Take-Away Exercise 2.
- **Lecture 10 (Network Problems - I)**: the Transshipment Problem is the direct generalisation of the Least-Cost Path Problem — both route flow through intermediate nodes under flow conservation; the difference is purely in the supply/demand pattern (one unit vs. many) and whether the destination is unique.

1.8 Moving Forward

- The LP module concludes the network-flow toolkit here. The course now turns to **Non-Linear Programming** — principles, exact solution methods (penalty method, barrier method, interior-point method), non-exact methods (local search, evolutionary computation, swarm intelligence), and applications to the Travelling Salesman Problem, Vehicle Routing

Problem, Facility Location Problem, and Network Design Problem — many of which build directly on the network representations developed in Lectures 10-11.

1.9 Further Reading

- Ahuja, R.K., Magnanti, T.L., and Orlin, J.B. (1993). *Network Flows: Theory, Algorithms, and Applications* — Chapter 9: The Transportation and Assignment Problems; Chapter 1: transshipment formulations.
- Winston, W.L. (2004). *Operations Research: Applications and Algorithms* — Chapter 7: Transportation, Assignment, and Transshipment Problems.
- Vanderbei, R.J. (2014). *Linear Programming: Foundations and Extensions* — Chapter 14: Network Flow Problems.
- SciPy documentation: `scipy.optimize.linprog` — docs.scipy.org